



Microcontroller programming

A microcontroller is a single-chip computer optimized for control tasks. A microcontroller is a microprocessor supplemented with peripherals integrated onto the circuit board. Nowadays, microcontrollers are found in many everyday devices, from digital thermometers to cars and even fast food toys.

In the past, microprocessor types were used to perform control tasks. When using microprocessors, additional ICs had to be built in due to the necessary peripherals. With the development of circuit technology, more and more peripherals could be integrated into the IC package, resulting in the microcontroller, which is a very compact circuit.

1. Introduction

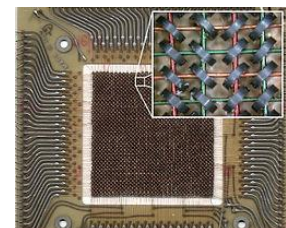
1.1. Computer generations

1.1.1. Computer Generation Zero: mechanical machines

The zero generation of computers includes the first computers in the modern sense, which contained signal receivers, also known as relays.

1.1.2. The first generation of computers: Electron tubes

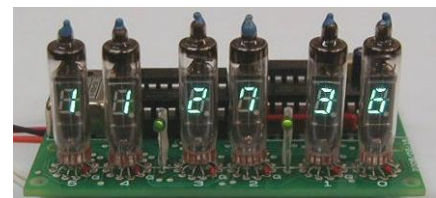
The first generation of computers lasted from 1946 to 1954, but its impact was felt until the end of the 1950s. The first generation of computers were vacuum tube-based digital machines. These computers were characterized by low operating speed, large size, low reliability, and high price. Their structure was processor-centric, with all data traffic passing through the processor. Only one type of operation could be performed at a time. The operating memory (also known as main memory or central memory) used vacuum tube circuits as storage elements. Peripherals were unique devices that varied from machine to machine.



1. Figure - Ferrite ring memory



2. Figure–Electron tubes



3. Figure - Nixie tubes - Numitron tubes - VFD tubes



1.1.3. The second generation of computers: the transistor

The second generation of computers lasted from around 1954 to 1964, following the invention of the transistor. Their reliability and operating speed increased compared to first-generation computers. A significant change was the emergence of channels that operated independently of the central processing unit. The structure of computers became memory-centric. In addition to traditional peripherals, magnetic disk and magnetic tape storage devices became widespread, enabling the storage and rapid transfer of large amounts of data and programs. Operational storage became more reliable, faster, and had greater capacity. Computer families appeared, consisting of "compatible" computers with different performance levels but identical programmability and using the same peripherals. The importance of programs, also known as software, increased.



4. Figure–Analog computer

1.1.4. The third generation of computers: integrated circuits, operational amplifiers, gate circuits

The third generation of computers lasted from 1964 to 1971. Instead of discrete semiconductor devices, they were built from integrated circuits. Compared to the previous generation, this meant another increase in speed and a reduction in size and power consumption. Computers belonging to this generation are characterized by the independent, parallel operation of their components. The DOS and OS operating systems were developed during this period.

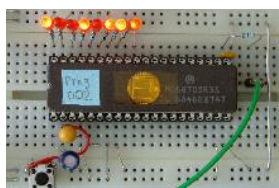
1.1.5. The fourth generation of computers: microprocessors and the spread of personal computers

The fourth generation of computers lasted from around 1971 to the mid-1990s. The advent of the world's first microprocessor is considered to mark the beginning of the fourth generation. It was characterized by the spread of microprocessors and personal computers, previously unimaginable operating speeds, storage density and capacity, and miniaturization. It made it possible to implement a computer's processor as a single component. New types of storage devices appeared (floppy disks, hard disks). Personal computers became mass-produced items. Special peripherals (mice, joysticks, scanners) helped to increase interactivity. The quality of printers reached the level of printing presses.

Computer networks and graphical operating systems became increasingly important. The range of applications for computers expanded significantly to include desktop publishing, animation, spreadsheet and database management, simulation, and expert systems.

1.1.6. The fifth generation of computers: multimedia and internet

The fifth generation of computers is characterized by the global spread of multimedia and the Internet. Terabyte storage devices, gigabyte operating memory, and screens with resolutions closer to that of the human eye have become commonplace. Interactive education provides a new tool for learning. The fifth generation will therefore be an era of new content in the dissemination, processing, and storage of information.

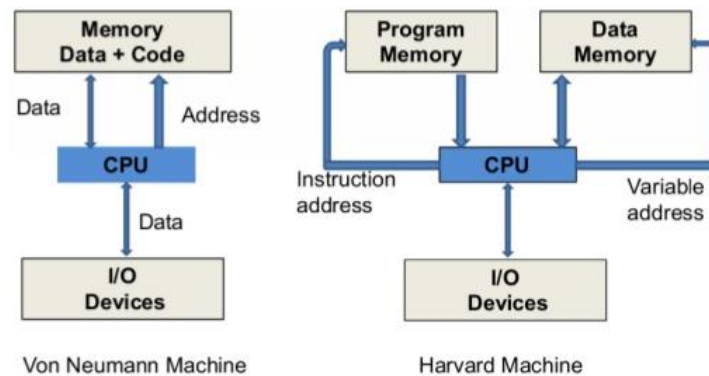


5. Figure - Microcontrollers and development environments

1.2. Computer hardware design

The most important feature of the **Neumann architecture** is that instruction code reading and arithmetic or logical operations cannot be performed simultaneously with data because the computer uses a single data bus. This is the bottleneck of the Neumann architecture.

The **Harvard architecture** is a design principle in which program code and data travel along separate, physically isolated paths to the processor.



6. Figure - Neumann és Harvard architectures

A system containing a central unit, memory, input/output units, and auxiliary circuits housed in a single case is called a single-chip microcomputer, also known as a **microcontroller**.

Most of these are Harvard architecture RISC (reduced instruction set computer) machines.

The instruction set coding is designed so that each instruction can be stored at a single program memory address.

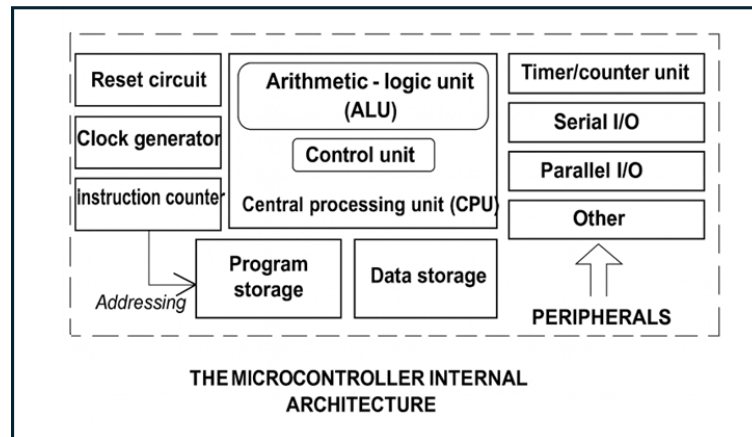
The result:

- Extremely fast operation (5 million instructions per second at a clock speed of 20 MHz)
- Extremely efficient use of program memory

In our exercises, we will be working with Microchip Technology's PICs (Peripheral Interface Controllers).



2. The internal design of a PIC



7. Figure–The internal design of a PIC

- Central processing unit (CPU), execution of instructions stored in the program memory
- Arithmetic logic unit (ALU)
- Control unit
- Clock generator
- Reset circuit (resets the system to its default state)
- Peripherals (connection to the outside world)
- Instruction counter (determines which register of the program memory to read the code containing the instruction from)

2.1. Clock solutions

In most cases, a digital system requires a clock signal to operate, which is generated by an oscillator circuit.

In the case of PIC controllers, there are various clock signal solutions to choose from:

- LP - Low Power Crystal (low power consumption quartz)
- XT - Quartz/ceramic resonator
- HS - High frequency quartz/ceramic resonator
- RC - Resistor - Capacitor
- Internal RC oscillator

Every PIC has two pins: OSC1 and OSC2, to which the clock signal generating components must be connected: An internal configuration bit pair in the case must be used to specify the type of clock signal configuration to be used.

2.2. RESET circuit

When switching on digital systems, it is necessary to set a well-defined default position, as this is the only way to ensure that the system operates correctly. This is the RESET process. When the power supply is switched on, the supply voltage rises and the oscillator starts to operate. With the PIC, a RESET can be triggered at any time by setting a separate pin (called MCLR) to a low and then a high level. Its use is not mandatory.



2.3. Programm memory

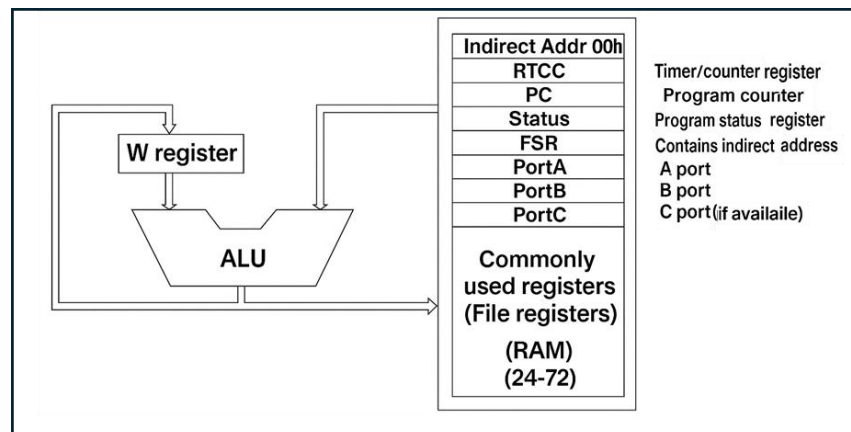
The central unit reads the instruction codes to be executed from the program memory. This memory can be of different types depending on various requirements: ROM, EPROM, EEPROM, and OTP.

- ROM (Read Only Memory)
- EPROM (Erasable Programmable Read Only Memory)
- EEPROM (Electrically Erasable Programmable Read Only Memory)
- OTP memory (One Time Programmable Memory)
- Flash memory (organized into special EEPROM blocks for faster access)

2.4. Data memory

During program execution, we perform operations on data. The registers containing the data—which make up the RAM memory—should be designed to be accessible to the central processing unit for quick access. In the case of the PIC, this is the RAM area called File Registers. Any register can be one of the operands of the operations performed by the ALU (Arithmetic and Logic Unit).

There are two types of registers here: fixed-address registers containing special data, which play an important role in the operation and programming of the controller, and general-purpose registers, which can be used to store arbitrary data. Program memory and data memory (RAM File Registers) are separate from each other. This is advantageous because the length of the data words (8 bits) does not have to match the length of the instruction words (12 bits wide).



8. Figure - PIC datamemory

2.5. WatchDog Timer (WDT)

During continuous operation, the microcontroller executes a cyclically repeating series of instructions. One instruction follows another, and if, for some external reason (circuit malfunction), an instruction is read incorrectly, this cyclicity ceases, and the running program "deviates" and does not return to the cycle.

In the watchdog circuit, an independently operating counter must be cleared with an instruction (CLRWDT) placed in the cyclic program. If this is not done, the counter expires and triggers a RESET process, which restarts the controller, meaning that the program can only "wander" until the counter (called the watchdog timer) expires. (Its use is not always necessary.)

2.6. Controller peripheral units

- Two-state input/output units – also known as digital I/O ports
- Timer/counter unit (RTCC)
- A/D converter (converts input analog signals into digital data)

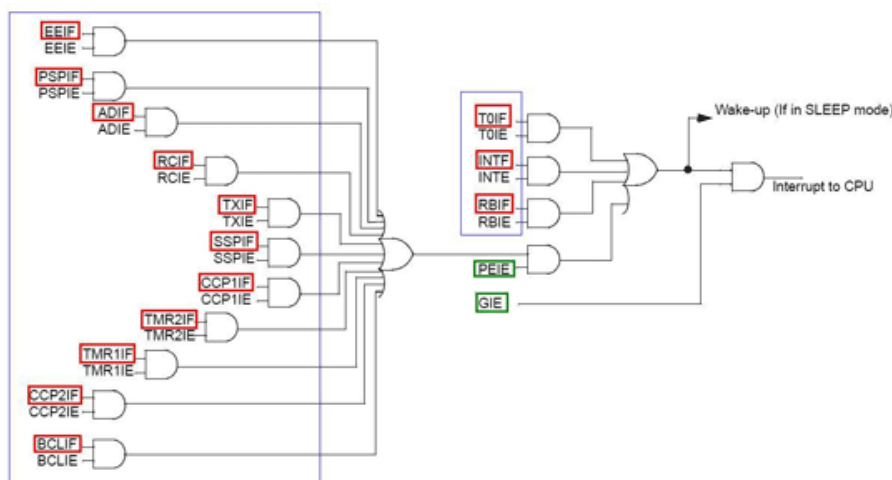


- PWM converter (converts input analog signals into digital data)
- Capture register (when an external signal appears, the value of the internal counter is written to this register)
- Compare register (the value written to this register is used to compare the value of the internal counter. It signals when the two values match)
- Serial peripheral (unit that implements serial data transfer)
- Other peripherals (SPI, I2C, USB, etc.)

2.7. Interruptions

If we want to detect the occurrence of an event with a controller, there are two ways to do this:

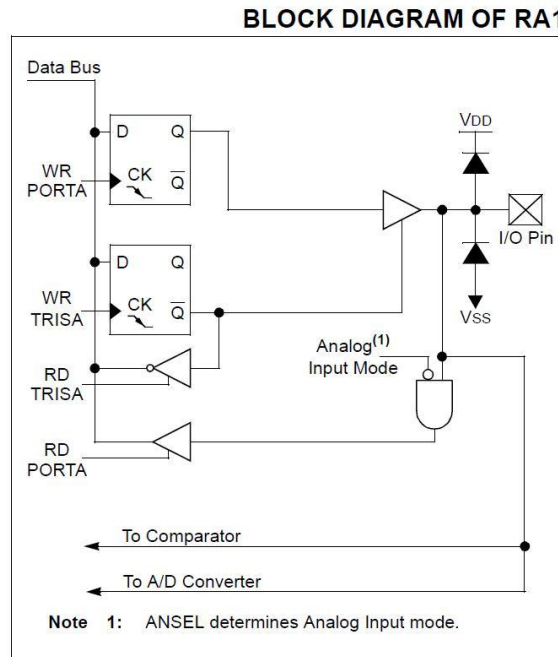
- We can detect the occurrence of external events by monitoring the status of an input.
- The processor itself signals a change in its status. (interrupt)



9. Figure - Simpler PIC interrupt system

2.8. I/O port structure

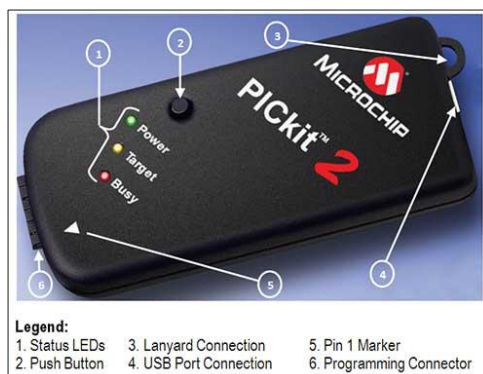
The square labeled I/O pin is a physical output accessible to the outside world, equipped with protective diodes. Information arrives or departs from the port labeled Data Bus, depending on whether it is configured as an output or input. The upper D register (simple buffer) reads in data on the clock signal and displays it at the Q output, which is sent to the output via a line driver if enabled by the other D register. The D register labeled TRIS sets the direction, which specifies whether it is an input or output in a given case. When '1' is written to the TRIS register, the port is configured as an input, and when '0' is written, it is configured as an output. The port is read using the RD PORT line driver. If the analog input is enabled, the route to the line driver is disabled, thus information can't be sent. The information can also be forwarded to other units, such as an A/D converter or comparator.



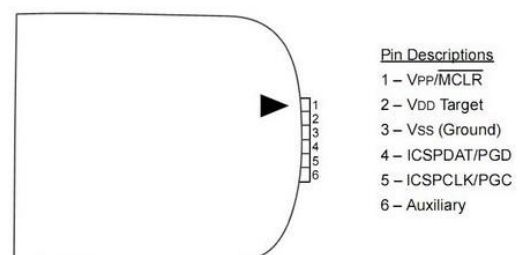
3. Program input, readout, and deletion options

Microchip offers two programming solutions: the faster parallel method and the simpler serial method. For serial programming, we will use PICkit2 in this workshop.

- Supports 8, 16, and 32-bit PIC microcontrollers
- USB connection (Fullspeed 12Mbps/s)
- Real-time program execution
- Built-in power surge and short-circuit protection
- Low voltage support (2.0V – 6.0V)
- Diagnostic LEDs (power, operation, error)
- Peripheral "freeze" at breakpoints



11. Figure - PICkit2 parts

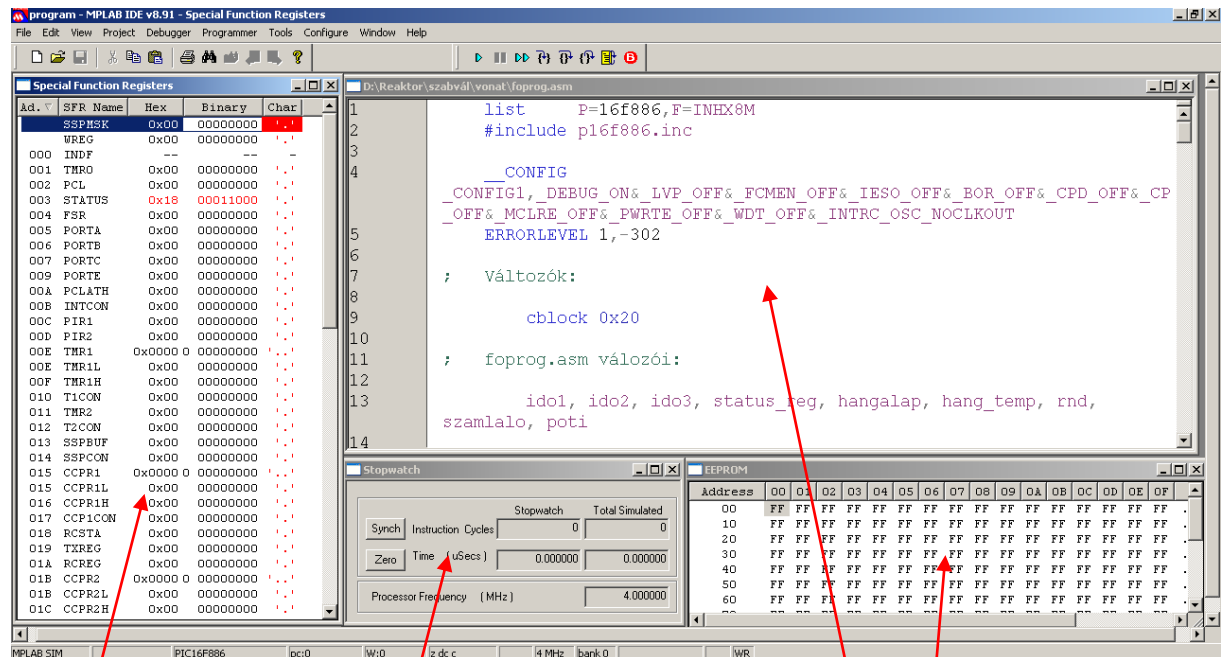


Note: The connector is a 6-pin header with 0.100" spacing and can accept 0.025" square pins.

12. Figure - PICkit2 pins



4. MPLAB IDE interface



Special registers

Stopwatch in simulator mode

Assembly/C level program code window

Data memory table

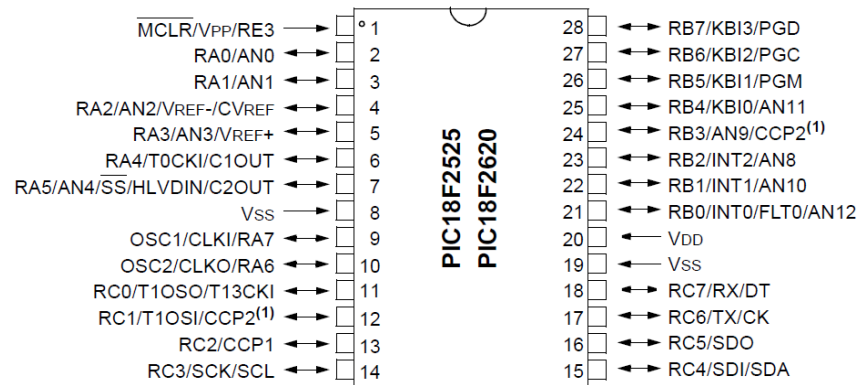
Before you start programming, connect the PICkit2 to your computer.

On your computer desktop, you will find a folder named M6_uC_prog. Copy this folder and rename the copy as follows: M6_uC_prog_12_34. (Replace 12 with the initials of your name and 34 with the initials of your colleague's name.)

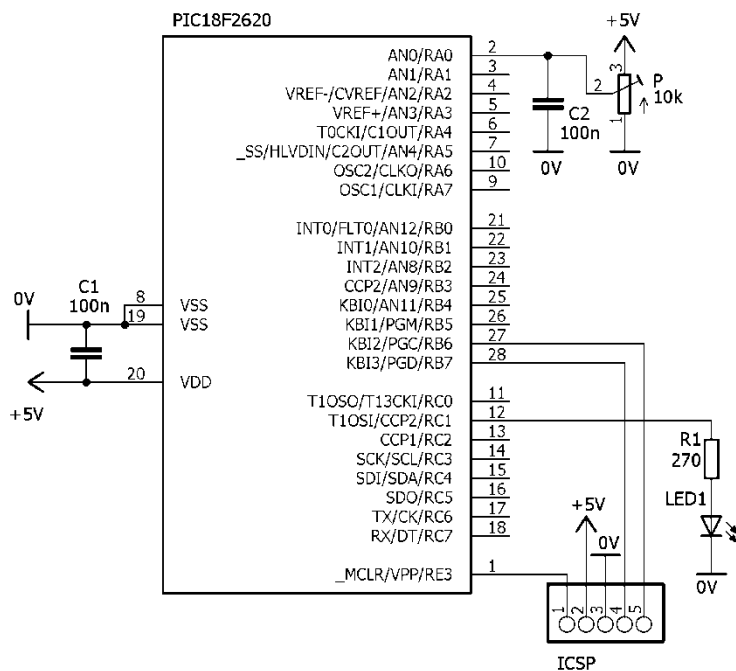


5. Measurement tasks

When programming the microcontroller, you need to create a simple circuit layout on a plug-in test panel and write the program for it. Below you can see the pin layout of the microcontroller used, which may be helpful when building the circuit.



13. Figure - PIC18F2525/2620 pinout diagram with pin layout



14. Figure – Measurement setup

Assemble the circuit shown in Figure 14. Start the MPLAB IDE program. Connect the PICKit2 (as shown in Figure 12) to the circuit via the ICSP connector. Use the 90° pin header as the ICSP connector. On the Project tab, use Open to open the frame.mcp file located in your own folder on your desktop. This will load the interface where you can create the program. The program will attempt to connect to the PICKit2. If everything is correct, the program will display a message indicating that it has recognized the microcontroller and will measure the supply voltage in the system. Then, in the Project menu, select Build All (Ctrl+F10). The code will then be "compiled" and, if there are no errors, loaded into the controller and the program will start. You will see the result: the LED will flash. The PICKit2 supplies power to the circuit; no external power supply is required!



1. Measure the LED current. You may only use an oscilloscope or multimeter in voltage measurement mode. Verify the result with calculations.
2. Measure the period and negative and positive time intervals of the square wave appearing at the output.
3. Currently, the timing of the output square wave in the program is done with delays. Change the value of the second delay to 25.000, compile the program (press F10), and measure the period of the signal that appears, as well as its negative and positive time intervals. Repeat the measurement, increasing the previously changed delay value to 100.000. Explain what you see.
4. In real-time systems, it is not customary to use delays for timing, because in this case the program cannot perform other tasks. Timing is achieved using a timer. To learn about the timer settings, open the microcontroller data sheet and follow the settings used in the code.
(PIC18F2620.pdf 11.0 TIMER0 MODULE)

In the initialization part of the program, rewrite 0 to 1 in the T0CONbits.TMR0ON = 0; line, thereby activating the Timer0 module. The Timer0 module is set to the timer function and any previous values in the timer are deleted here.

At while(1), delete the part between the curly brackets and insert the following code:

```
if (INTCONbits.TMR0IF)
{
    LED1 = !LED1;
    INTCONbits.TMR0IF = 0;
}
```

Compile the program (F10) and observe what happens using the oscilloscope.

Explain what you see and describe why this solution is more optimal than the one you learned in the previous task.

5. Rewrite T0CON = 0b10000011; to T0CON = 0b10000100;. Compile the program and interpret what you see. Event handling is more convenient with interrupts, but this is not part of the course. When you are done with this task, turn off the Timer0 module.
6. Using a PWM (pulse width modulation) generator. Similar to the previous task, our goal is to generate a 50% duty cycle square wave for the LED. In the initialization section, change the 0 in the line T2CONbits.TMR2ON = 0; to 1 to enable the PWM module's time base, then compile the program with F10. In this task, connect the oscilloscope probe to pin RC1, which is also one of the outputs of the PWM module, then measure the frequency and positive duty cycle of the signal at the output. To learn about the settings of the PWM module, open the microcontroller data sheet and follow the settings used in the code.

(PIC18F2620.pdf 15.4 PWM MODE)

Currently, the system clock is $F_{osc} = 8 \text{ MHz}$. The Timer2 division is 16. Calculate the output frequency based on the data sheet.



7. Change the CCPR2L value from the current 125 to 25 (F10), then to 225 (F10). What do you observe? Verify the measured fill factor values by calculation. The next task is to use the 10-bit A/D converter (analog-to-digital converter) in the PIC. To learn about the settings of the A/D module, open the microcontroller data sheet and follow the settings used in the code.

(19.0 10-BIT ANALOG-TO-DIGITAL CONVERTER (A/D) MODULE)

To enable the module, set the 0 to 1 in the line `ADCON0bits.ADON = 0;` in the Initialization section. Then, between the curly braces of the `while(1)` loop, type the following: `AD_conv();`

This calls the `AD_conv` function, which starts the conversion and reads the result from the converter. Set the 10 k Ω potentiometer to its middle position, which you can check with a multimeter.

The conversion result can determine the duty cycle of the PWM generator. To do this, the 10-bit AD value must be adjusted to the maximum duty cycle value of the PWM generator, which is 250. This means that the conversion result must be divided by 4.1.

Enter the following line after the `AD_conv();` function call: `CCPR2L = AD_value/4.1;`

Compile the program (F10) and observe the brightness of the LED when changing the potentiometer value.

8. Examination of digitized values. For 11 predefined voltage values, examine the duty cycle value, which you also verify by calculation in the report.

Voltage input [V]	0	0.5	1	1.5	2	2.5	3	3.5	4	4.5	5
Positive intervals [%]											